Interface in Java

https://www.javatpoint.com/difference-between-abstract-class-and-interface

Interface

- An **interface in java** is a blueprint of a class.
- It has static constants and abstract methods.
- There can be only abstract methods in the Java interface, not method body.
- It is used to achieve abstraction and multiple inheritance in Java.

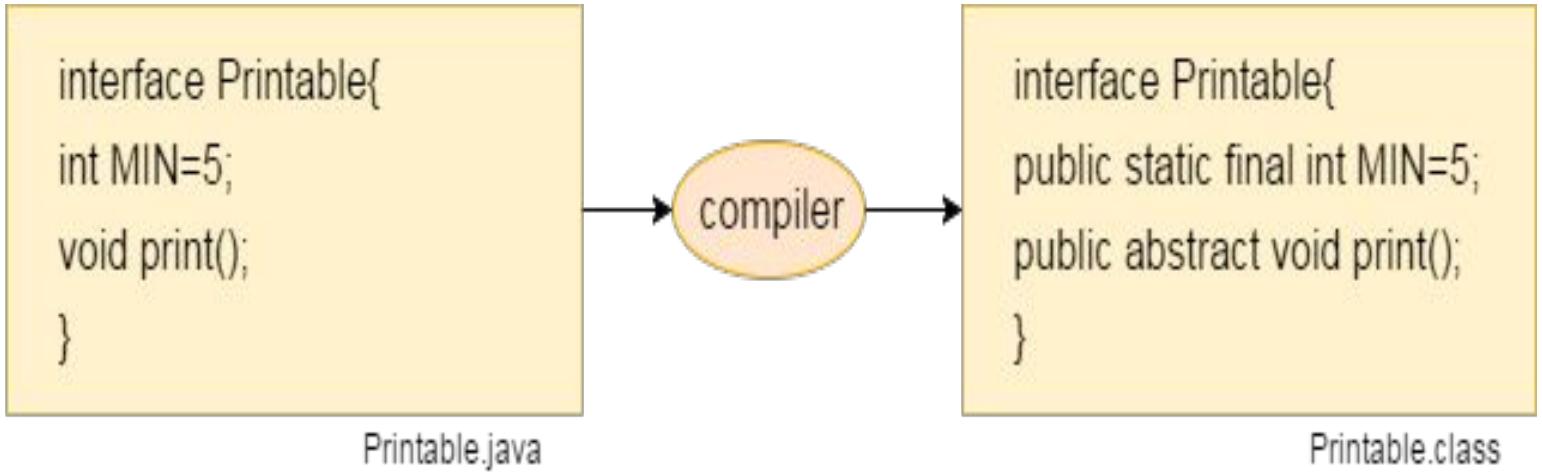
Why use Java interface?

- There are mainly three reasons to use interface. They are given below :

1. It is used to achieve abstraction.
2. By interface, we can support the functionality of multiple inheritance.

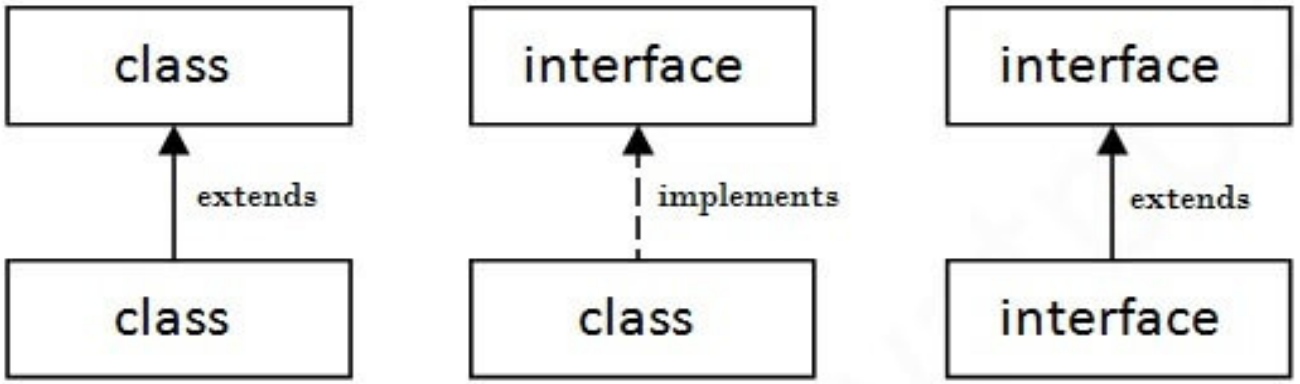
Internal addition by the compiler

- The Java compiler adds public and abstract keywords before the interface method.
- Moreover, it adds public, static and final keywords before data members.



The relationship between classes and interfaces

- a class extends another class, an interface extends another interface, but a **class implements an interface**.



Java Interface Example1

```
interface A{
void print();
}
class Test implements A
{
public void print()
{System.out.println("Hello");}

public static void main(String args[])
  {
Test obj = new Test();
obj.print();
  }
}
```

Output:
Hello

Java Interface Example2

```
interface A{
void draw();
}
//Implementation: by second user
class Rectangle implements A{
public void draw(){System.out.println("Drawing rectangle");}
}
class Circle implements A{
public void draw(){System.out.println("Drawing circle");}
}

class TestInterface{
public static void main(String args[]){
A d=new Circle();
A e=new Rectangle ();
d.draw();
e.draw();
}}
```

Output:
Drawing circle
Drawing rectangle

Java Interface Example2

- In previous example, the A interface has only one method. Its implementation is provided by Rectangle and Circle classes.
- In a real scenario, an interface is defined by someone else, but its implementation is provided by different implementation providers.
- Moreover, it is used by someone else. The implementation part is hidden by the user who uses the interface.

Java Interface Example3

```
interface Salary{
   float salary();
   }
   class CEO implements Salary{
   public float salary(){return 50000;}
   }
   class Manager implements Salary{
   public float salary(){return 30000;}
   }

   class Employee implements Salary{
   public float salary(){return 20000;}
   }
```

Output:

Salary of CEO: 50000.0
Salary of Manager: 30000.0
Salary of Employee20000.0

```
class SalaryInterface
{
  public static void main(String[] args)
{
Salary a =new CEO();
System.out.println("Salary of CEO:
"+a.salary());

Salary b =new Manager();
System.out.println("Salary of Manager:
"+b.salary());

Salary c =new Employee();
System.out.println("Salary of
Employee"+c.salary());

    }}
```

Multiple inheritance in Java by interface

- If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

Java Interface Example4

```
interface A{
   void print();
   }
   interface B{
   void show();
   }
   class MultiInterface implements A,B
{
   public void print()
{System.out.println("Hello from A");}
   public void show()
{System.out.println("Welcome from
    B");}
public static void
    main(String args[])
{
   MultiInterface obj = new
    MultiInterface ();
   obj.print();
   obj.show();
    }
}
```

Multiple inheritance is not supported through class in java, but it is possible by an interface, why?

- Multiple inheritance is not supported in the case of class because of ambiguity.
- However, it is supported in case of an interface because there is no ambiguity.
- It is because its implementation is provided by the implementation class.
- In the next example, A and B interface have same methods but its implementation is provided by class **MultiInterface1** , so there is no ambiguity.

Java Interface Example5

```
 interface A
{
   void print();
}
   interface B
{
   void print();
   }

   class MultiInterface1  implements A,B
{
   public void print()
   {System.out.println("Hello");}

   public static void main(String args[])
   {
   MultiInterface1 obj = new MultiInterface1();
   obj.print();
    }
   }
```

Output:
Hello

Interface inheritance Example

- A class implements an interface, but one interface extends another interface.

Java Interface Example6

```
interface A{
   void print();
   }
   interface B extends A{
   void show();
   }
   class InterfaceInheritance implements B{
   public void print(){System.out.println("Hello");}
   public void show(){System.out.println("Welcome");}

   public static void main(String args[]){
   InterfaceInheritance obj = new InterfaceInheritance();
   obj.print();
   obj.show();
    }
   }
```

Output:
Hello
Welcome

Java Interface Example6

```
interface A{
   void print();
   }
   interface B extends A{
   void show();
   }
   class InterfaceInheritance implements B{
   public void print(){System.out.println("Hello");}
   public void show(){System.out.println("Welcome");}

   public static void main(String args[]){
   InterfaceInheritance obj = new InterfaceInheritance();
   obj.print();
   obj.show();
    }
   }
```

Output:
Hello
Welcome

default method in Interface

we can have method body in interface. But we need to make it default method. Let's see an example:

```
interface A{
  void draw();
  default void msg(){System.out.println("default method");}
  }
  class B implements A{
  public void draw(){System.out.println("drawing rectangle");}
  }
  class InterfaceDefault{
  public static void main(String args[]){
  A d=new B();
  d.draw();
  d.msg();
  }}
```

Output:
drawing rectangle
default method

Satic method in Interface

Since Java 8, we can have static method in interface. Let's see an example:

```
interface A{
  void draw();
  static int cube(int x){return x*x*x;}
  }
  class B implements A{
  public void draw(){System.out.println("drawing rectangle");}
  }

  class InterfaceStatic{
  public static void main(String args[]){
  A d=new B();
  d.draw();
  System.out.println(A.cube(3));
  }}
```

Output:
drawing rectangle
27

Final Example

```
import java.util.Scanner;
interface Calculator{
        final static double x=5,y=6;
        abstract double min(double e, double z);
        abstract double multi(double e, double z);
        abstract double div(double e, double z);
}
interface Summation
{
  double sum(double a, double b);
}
```

Final Example

```
class InterfaceImplementedClass implements Calculator,Summation
{
 double x,y;
 void hello(){System.out.println("This is Interface Implemented Class ");}
 public double sum(double a, double b)
        {x = a;
         y = b;
         double c = x+y;
         return c;}
 public double multi(double a, double b)
       {x = a;
        y = b;
        double c = x*y;
        return c; }
```

Final Example

```
public double min(double a, double b)
      { x = a;
        y = b;
        double c = x-y;
        return c;}
public double div(double a, double b)
      {x = a;
            y = b;
       double c = x/y;
      return c;}
}
```

Final Example

```
class InterfaceExam
{
 public static void main(String [] args) {
 InterfaceImplementedClass c = new InterfaceImplementedClass();
 Scanner s = new Scanner(System.in);

 System.out.println("Enter Any Value:");
 double num1 = s.nextDouble();

 System.out.println("Enter Another Value:");
 double num2 = s.nextDouble();
 c.hello();
double temp = c.sum(num1,num2);
System.out.println("Summation of a and b :"+temp);
```

Final Example

```
double temp1 = c.multi(num1,num2);
System.out.println("Multiplication of a and b :"+temp1);

double temp2 = c.min(num1,num2);
System.out.println("Minus of a and b :"+temp2);

double temp3 = c.div(num1,num2);
System.out.println("Division of a and b :"+temp3);
 }

}
```